# PHP

## POO & PDO — Résumé Complet + Correction des Exercices

Classes · Héritage · Encapsulation · PDO · CRUD · Sessions · Cookies

Développement Web : PHP — Ateliers 1 à 5

# PARTIE 1 — PHP : LES BASES

## 1. Variables, Types & Constantes

| Déclaration | Exemple | Remarque |
|---|---|---|
| `$var` | `$a = 12; $b = 'hello';` | Préfixe $ obligatoire, pas de déclaration de type |
| `define` | `define("PI", 3.14);` | Constante — valeur immuable |
| `const` | `const MAX = 100;` | Constante dans une classe ou globale |

| Type | Exemple | Vérification |
|---|---|---|
| integer | `$n = 42;` | `is_integer($n)` |
| float | `$f = 3.14;` | `is_float($f)` |
| string | `$s = "Bonjour";` | `is_string($s)` |
| boolean | `$b = true;` | `is_bool($b)` |
| array | `$t = array(1,"a");` | `is_array($t)` |
| null | `$x = null;` | `is_null($x)` |
| object | `$o = new Classe();` | `is_object($o)` |

### Superglobales essentielles

| Variable | Rôle |
|---|---|
| `$_POST['champ']` | Données envoyées via méthode POST |
| `$_GET['champ']` | Données envoyées via méthode GET (URL) |
| `$_SESSION['cle']` | Variables de session (persistantes entre pages) |
| `$_COOKIE['cle']` | Cookies stockés côté client |
| `$_FILES['champ']` | Fichiers uploadés |
| `$_SERVER['PHP_SELF']` | Nom du script courant |

## 2. Structures de contrôle & Boucles

```
// if / elseif / else
if ($note >= 10) { echo 'Admis'; }
elseif ($note >= 8) { echo 'Rattrapage'; }
else { echo 'Refusé'; }

// switch
switch ($op) {
  case '+': echo $a+$b; break;
  default:  echo 'Inconnu';
}

// for
for ($i=0; $i<10; $i++) { echo $i.' '; }

// while
while ($i < 5) { echo $i; $i++; }
```

```
// foreach sur tableau indicé
foreach ($tab as $val) { echo $val; }

// foreach sur tableau associatif
foreach ($tab as $cle => $val) { echo "$cle : $val"; }
```

## 3. Tableaux

```
// Tableau indicé
$notes = array(15, 12, 18, 9);
$notes[] = 14;             // ajoute à la fin
echo count($notes);        // 5

// Tableau associatif
$etudiant = array('nom'=>'Ali', 'moyenne'=>15.5);
echo $etudiant['nom'];     // Ali

// Tableau multidimensionnel
$etudiants = array(
  'Ali'   => array('ville'=>'Tunis', 'age'=>21),
  'Sarra' => array('ville'=>'Sfax',  'age'=>22)
);

// Fonctions utiles
count($tab)              // taille
in_array($val, $tab)     // recherche booléenne
array_search($val, $tab) // retourne la clé
sort($tab)               // tri croissant
ksort($tab)              // tri par clés
implode(',', $tab)       // tableau -> chaîne
explode(',', $chaine)    // chaîne -> tableau
```

## 4. Fonctions

```
// Déclaration
function calculMoyenne($notes, $coef=1) {
  $somme = 0;
  foreach ($notes as $n) $somme += $n;
  return ($somme / count($notes)) * $coef;
}

// Passage par référence
function incrementer(&$nb) { $nb += 1; }
$x = 5; incrementer($x); // $x vaut maintenant 6

// Appel
$moy = calculMoyenne(array(15, 12, 18)); // 15
$moy2 = calculMoyenne(array(15, 12), 2); // avec coef
```

## 5. Formulaires

```
<!-- Formulaire HTML -->
<form action='traitement.php' method='post'>
  <input type='text'     name='nom'>
  <input type='radio'    name='sexe' value='H'> H
```

```
  <input type='radio'    name='sexe' value='F'> F
  <input type='checkbox' name='comp[]' value='PHP'> PHP
  <input type='checkbox' name='comp[]' value='JS'>  JS
  <select name='ville'>
    <option>Tunis</option>
    <option>Sfax</option>
  </select>
  <input type='submit' value='Envoyer'>
</form>
```

```
<?php
// Récupération POST
$nom  = $_POST['nom'];
$sexe = $_POST['sexe'];
// Cases à cocher (tableau)
foreach ($_POST['comp'] as $c) echo $c . ' ';
// Sécurité : vérifier existence
if (isset($_POST['nom']) && !empty($_POST['nom'])) { ... }
?>
```

# PARTIE 2 — POO : PROGRAMMATION ORIENTÉE OBJET

## 1. Classe, Attributs, Méthodes

```php
<?php
class Personne {
  // Attributs (encapsulation)
  private $nom;
  private $prenom;
  private $age;

  // Constructeur
  public function __construct($nom='user', $prenom='', $age=0) {
    $this->nom    = $nom;
    $this->prenom = $prenom;
    $this->age    = $age;
  }

  // Getters / Setters
  public function getNom()    { return $this->nom; }
  public function setNom($n)  { $this->nom = $n; }
  public function getPrenom() { return $this->prenom; }
  public function getAge()    { return $this->age; }

  // Méthode
  public function affiche() {
    echo $this->nom . ', ' . $this->prenom . ', ' . $this->age . ' ans';
  }

  // Destructeur
  public function __destruct() { /* libérer ressources */ }
}
?>
```

## 2. Instanciation & Utilisation

```php
$p1 = new Personne('Ali', 'Ben Salem', 21);
$p2 = new Personne();          // valeurs par défaut

$p1->affiche();                // Ali, Ben Salem, 21 ans
$p1->setNom('Ahmed');          // modificateur
echo $p1->getNom();            // Ahmed
```

## 3. Modificateurs d'accès

| Modificateur | Accessible depuis |
|---|---|
| `public` | Partout : classe, sous-classe, code externe |
| `protected` | Classe elle-même + classes filles (héritage) |
| `private` | Uniquement dans la classe déclarante |

## 4. Héritage (extends)

```php
class Etudiant extends Personne {
  private $moyenne;
```

```
  public function __construct($nom, $prenom, $age, $moy) {
    parent::__construct($nom, $prenom, $age); // appel du constructeur parent
    $this->moyenne = $moy;
  }

  public function getMoyenne() { return $this->moyenne; }
  public function setMoyenne($m) { $this->moyenne = $m; }

  // Redéfinition de méthode
  public function affiche() {
    parent::affiche();    // appel méthode parente
    echo ', moyenne: ' . $this->moyenne;
  }
}

$e = new Etudiant('Ali', 'Ben', 21, 15.5);
$e->affiche();  // Ali, Ben, 21 ans, moyenne: 15.5
```

## 5. Membres statiques

```
class Voiture {
  private static $nbRoues = 4;

  public static function getRoues() {
    return self::$nbRoues;  // self:: pour accès statique interne
  }
}

echo Voiture::getRoues();  // 4 (sans instanciation)
```

## 6. Interface & Polymorphisme

```
interface Animal {
  public function parler();
}

class Chat implements Animal {
  public function parler() { echo 'Miaou'; }
}

class Chien implements Animal {
  public function parler() { echo 'Ouaf'; }
}

// Polymorphisme
$animaux = array(new Chat(), new Chien());
foreach ($animaux as $a) { $a->parler(); } // Miaou Ouaf
```

# PARTIE 3 — PDO : ACCÈS BASE DE DONNÉES + CRUD

## 1. Connexion PDO

```php
<?php  // connexion.php
$servername = 'localhost';
$username   = 'root';
$password   = '';
$dbname     = 'ma_base';

try {
  $cnx = new PDO(
    "mysql:host=$servername;dbname=$dbname",
    $username,
    $password
  );
  $cnx->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
  echo 'Connexion échouée : ' . $e->getMessage();
}
?>
```

## 2. CRUD — CREATE (INSERT)

### Sans requête préparée

```php
require_once('connexion.php');
$req = "INSERT INTO Client(nom, cin, date_naissance)
         VALUES('bilel', 11111111, '1990-05-12')";
$result = $cnx->exec($req);
if ($result == 0) echo 'Échec'; else echo 'Succès';
```

### Avec requête préparée (recommandé — protège des injections SQL)

```php
// Récupération données formulaire
$nom = $_POST['nom'];
$cin = $_POST['cin'];
$dn  = $_POST['dn'];

// Méthode 1 : marqueurs nommés + tableau associatif
$req = $cnx->prepare(
  'INSERT INTO Client(nom, cin, date_naissance) VALUES(:nom, :cin, :dn)'
);
$tab = array(':nom'=>$nom, ':cin'=>$cin, ':dn'=>$dn);
$result = $req->execute($tab);

// Méthode 2 : marqueurs interrogatifs + bindParam
$req = $cnx->prepare(
  'INSERT INTO Client(nom, cin, date_naissance) VALUES(?, ?, ?)'
);
$req->bindParam(1, $nom, PDO::PARAM_STR);
$req->bindParam(2, $cin, PDO::PARAM_INT);
$req->bindParam(3, $dn);
$result = $req->execute();

if ($result) echo 'Ajouté avec succès';
else         echo 'Échec d\'ajout';
```

## 3. CRUD — READ (SELECT)

```
require_once('connexion.php');

// Sélection de tous les enregistrements
$stmt = $cnx->query('SELECT * FROM Client');

// fetch() : ligne par ligne
while ($row = $stmt->fetch()) {
  echo $row['id'] . ' ' . $row['nom'] . '<br>';
}

// fetchAll() : tout en une fois
$stmt    = $cnx->query('SELECT * FROM Client');
$clients = $stmt->fetchAll();
foreach ($clients as $c) {
  echo $c['nom'] . ' ' . $c['date_naissance'] . '<br>';
}

// Modes de récupération
$row = $stmt->fetch(PDO::FETCH_ASSOC); // tableau associatif
$row = $stmt->fetch(PDO::FETCH_NUM);   // tableau indicé
$row = $stmt->fetch(PDO::FETCH_OBJ);   // objet -> $row->nom

// Compter les résultats
echo $stmt->rowCount();    // nombre de lignes
echo $stmt->columnCount(); // nombre de colonnes
```

### SELECT avec paramètre (requête préparée)

```
$id  = $_GET['id'];
$req = $cnx->prepare('SELECT * FROM Client WHERE id = :id');
$req->execute(array(':id' => $id));
$client = $req->fetch();
echo $client['nom'];
```

## 4. CRUD — UPDATE

```
require_once('connexion.php');

// Sans requête préparée
$req    = "UPDATE Client SET nom='ridha' WHERE id=1";
$result = $cnx->exec($req);

// Avec requête préparée
$nom = $_POST['nom'];
$id  = $_POST['id'];
$req = $cnx->prepare('UPDATE Client SET nom=:nom WHERE id=:id');
$req->execute(array(':nom'=>$nom, ':id'=>$id));

if ($result) echo 'Modifié avec succès';
else         echo 'Échec de modification';
```

## 5. CRUD — DELETE

```
require_once('connexion.php');

// Sans requête préparée
```

```php
$req    = 'DELETE FROM Client WHERE cin = 11111111';
$result = $cnx->exec($req);

// Avec requête préparée
$id  = $_GET['id'];
$req = $cnx->prepare('DELETE FROM Client WHERE id = :id');
$req->execute(array(':id' => $id));

if ($result) echo 'Supprimé avec succès';
else         echo 'Échec de suppression';
```

## 6. Transactions

```php
$cnx->beginTransaction();
try {
  $req1 = $cnx->prepare('INSERT INTO Compte(solde,type) VALUES(?,?)');
  $req1->execute(array(100, 'epargne'));
  $req2 = $cnx->prepare('INSERT INTO Compte(solde,type) VALUES(?,?)');
  $req2->execute(array(500, 'courant'));
  $cnx->commit();
  echo 'Transaction réussie';
} catch (Exception $e) {
  $cnx->rollBack();
  echo 'Transaction annulée : ' . $e->getMessage();
}
```

## PARTIE 4 — CLASSE AVEC PDO : PATTERN COMPLET

Ce pattern montre comment intégrer PDO directement dans une classe PHP (Atelier 5 — Correction complète).

### personne.php — Classe Personne complète avec CRUD

```
<?php
class Personne {
  private $nom;
  private $prenom;
  private $age;
  private $sexe;
  private $ville;
  private $competence;
  private $photo;

  public function __construct($nom='user', $prenom='', $age=0,
                              $sexe='', $ville='', $comp='', $photo='') {
    $this->nom        = $nom;
    $this->prenom     = $prenom;
    $this->age        = $age;
    $this->sexe       = $sexe;
    $this->ville      = $ville;
    $this->competence = $comp;
    $this->photo      = $photo;
  }

  // ■■■ CREATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  public function insertion($cnx) {
    $req = 'INSERT INTO personne
            VALUES(null, :nom, :prenom, :age, :sexe, :ville, :comp, :photo)';
    $params = array(
      ':nom'=>$this->nom, ':prenom'=>$this->prenom,
      ':age'=>$this->age, ':sexe'=>$this->sexe,
      ':ville'=>$this->ville, ':comp'=>$this->competence,
      ':photo'=>$this->photo
    );
    $reqpre = $cnx->prepare($req);
    $reqpre->execute($params);
  }

  // ■■■ READ (tous) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  public function selection($cnx) {
    $req = $cnx->prepare('SELECT * FROM personne');
    $req->execute();
    return $req; // retourne PDOStatement
  }

  // ■■■ READ (par id) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  public function selection_id($cnx, $id) {
    $req = $cnx->prepare('SELECT * FROM personne WHERE id=:id');
    $req->execute(array(':id'=>$id));
    $row = $req->fetch();
    // Remplir l'objet courant
    $this->nom        = $row['nom'];
    $this->prenom     = $row['prenom'];
    $this->age        = $row['age'];
```

```
    $this->sexe       = $row['sexe'];
    $this->ville      = $row['ville'];
    $this->competence = $row['competences'];
    $this->photo      = $row['photo'];
  }

  // ■■■ UPDATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  public function modification($cnx, $id) {
    $req = 'UPDATE personne
            SET nom=:nom, prenom=:prenom, age=:age, sexe=:sexe,
                ville=:ville, competences=:comp, photo=:photo
            WHERE id=:id';
    $params = array(
      ':nom'=>$this->nom, ':prenom'=>$this->prenom,
      ':age'=>$this->age, ':sexe'=>$this->sexe,
      ':ville'=>$this->ville, ':comp'=>$this->competence,
      ':photo'=>$this->photo, ':id'=>$id
    );
    $cnx->prepare($req)->execute($params);
  }

  // ■■■ DELETE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  public function suppression($cnx, $id) {
    $req = $cnx->prepare('DELETE FROM personne WHERE id=:id');
    $req->execute(array(':id'=>$id));
  }

  // ■■■ Getters / Setters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  public function getNom()       { return $this->nom; }
  public function setNom($n)     { $this->nom = $n; }
  public function getPrenom()    { return $this->prenom; }
  public function getAge()       { return $this->age; }
  public function getSexe()      { return $this->sexe; }
  public function getVille()     { return $this->ville; }
  public function getCompetence(){ return $this->competence; }
  public function getPhoto()     { return $this->photo; }
}
?>
```

## index.php — Formulaire + Affichage + Suppression

```
<?php
require 'personne.php';
require 'connexion.php';   // fournit $cnx
$p = new Personne();

// ■■■ Enregistrement ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if (isset($_POST['enregistrer'])) {
  $comp = '';
  if (isset($_POST['comp']))
    foreach ($_POST['comp'] as $c) $comp .= $c . ' ';
  $photo = $_FILES['photo']['name'];
  move_uploaded_file($_FILES['photo']['tmp_name'], 'images/'.$photo);
  $p = new Personne($_POST['nom'], $_POST['prenom'], $_POST['age'],
                    $_POST['rad'], $_POST['ville'], $comp, $photo);
  $p->insertion($cnx);
```

```
}

// ■■■ Suppression ■■■
```

```
  header('Location: index.php');
}
?>
<form method='post'>
  Nom:    <input type='text' name='nom' value='<?= $p->getNom() ?>'><br>
  Prénom: <input type='text' name='prenom' value='<?= $p->getPrenom() ?>'><br>
  Age:    <input type='number' name='age' value='<?= $p->getAge() ?>'><br>
  <input type='submit' name='modifier' value='Modifier'>
</form>
```

# PARTIE 5 — COOKIES & SESSIONS

## 1. Cookies

| Action | Code |
|---|---|
| Créer | `setcookie('Langue', 'fr', time()+86400, '/', 'site.tn', 0);` |
| Lire | `if (isset($_COOKIE['Langue'])) echo $_COOKIE['Langue'];` |
| Modifier | `$n = $_COOKIE['Visites']+1; setcookie('Visites', $n, time()+86400);` |
| Supprimer | `setcookie('Langue', '', time()-3600, '/', 'site.tn', 0);` |

■ *setcookie() doit être appelé AVANT tout affichage HTML (avant echo, print…)*

## 2. Sessions

| Action | Code |
|---|---|
| Démarrer | `session_start();  // début de chaque page` |
| Sauvegarder | `$_SESSION['user'] = 'Ali';` |
| Lire | `if (isset($_SESSION['user'])) echo $_SESSION['user'];` |
| Supprimer var | `unset($_SESSION['user']);` |
| Détruire tout | `session_destroy();` |
| ID de session | `echo session_id(); echo session_name();` |

■ *session_start() doit être la première instruction de chaque page qui utilise les sessions.*

## 3. Différences Cookie vs Session

| Critère | Cookie | Session |
|---|---|---|
| Stockage | Côté client (navigateur) | Côté serveur |
| Durée | Définie (expire) | Jusqu'à fermeture navigateur ou session_destroy() |
| Sécurité | Moins sécurisé | Plus sécurisé |
| Taille | ~4 Ko max | Illimitée |
| Usage typique | Préférences, thème, langue | Authentification, panier, données sensibles |

# PARTIE 6 — CORRECTION DES ATELIERS

## Atelier N°1 — Bases PHP

### Exercice 1 — Conversions de types & isset()

```php
<?php
// Résultat du code de conversion de types :
// $truc = '5'     → string
// $truc = '5'.'2' = '52'  → string
// $truc = '52'*5  = 260   → integer
// $truc = 260.'2' = '2602'→ string
// $truc = '2602'/100 = 26.02 → double (float)

// isset()
$s = 'test';
echo isset($s); // 1 (true)
echo isset($j); // rien affiché (false)
```

### Exercice 2 — Afficher la date et l'heure

```php
<?php
$date  = date('d-m-Y');
$heure = date('H:i');
echo 'Nous sommes le ' . $date . ' et il est ' . $heure;
?>
```

### Exercice 3 — Manipulation de chaînes

```php
<?php
$chaine = "<p>Nous allons tester\nquelques <em>exemples</em> de fonctions\n"
        . "de manipulation de chaînes de caractères.</p>";

// 1. Fonctions de manipulation
echo nl2br($chaine);          // convertit \n en <br>
echo strip_tags($chaine);     // supprime les balises HTML
echo strtoupper($chaine);     // tout en majuscules
echo strrev($chaine);         // inverse la chaîne
echo ucwords($chaine);        // 1ère lettre de chaque mot en maj

// 2. N/10 premiers caractères
$N = strlen($chaine);
for ($i = 0; $i < $N/10; $i++) echo $chaine[$i];

// 3. Sous-chaîne du 6ème caractère sur 6
echo substr($chaine, 5, 6);

// 4. Remplacer em> par strong>
echo str_replace('em>', 'strong>', $chaine);
?>
```

### Exercice 4 — Afficher chaque lettre sur une ligne

```php
<?php
$chaine = 'Bonjour';
for ($i = 0; $i < strlen($chaine); $i++)
  echo $chaine[$i] . '<br>';
?>
```

### ■ Exercice 5 — Table de multiplication

```php
<?php
echo '<table border=1><tr><td></td>';
for ($j = 2; $j <= 9; $j++) echo "<td><b>$j</b></td>";
echo '</tr>';
for ($i = 2; $i <= 9; $i++) {
  echo "<tr><td><b>$i</b></td>";
  for ($j = 2; $j <= 9; $j++) {
    $prod = $i * $j;
    $bold = ($i == $j) ? "<b>$prod</b>" : $prod;
    echo "<td>$bold</td>";
  }
  echo '</tr>';
}
echo '</table>';
?>
```

### ■ Exercice 6 — Fonctions debutPage() / finPage() + include

**mesfonctions.php**

```php
<?php
function debutPage() {
  echo '<!DOCTYPE html><html><head>';
  echo '<meta charset="utf-8"><title>Ma Page</title></head><body>';
}
function finPage() {
  echo '</body></html>';
}
?>
```

**bonjour.php**

```php
<?php
include 'mesfonctions.php';
debutPage();
echo '<h1>Bienvenue sur mon site !</h1>';
finPage();
?>
```

## Atelier N°2 — Tableaux, Fonctions, Formulaires

### ■ Exercice 1 — Parcours d'un tableau

```php
<?php
$tab = array('bonjour1','bonjour2','bonjour3','bonjour4','bonjour5','bonjour6');

// Avec for
for ($i = 0; $i < count($tab); $i++)
  echo $tab[$i] . '<br>';

// Avec foreach
foreach ($tab as $val)
  echo $val . '<br>';
?>
```

### ■ Exercice 2 — Tableaux multidimensionnels associatifs

```php
<?php
// 1. Clés=noms, valeurs=tableaux indicés
```

```php
$personnes = array(
  'Dupont' => array('Paul', 'Paris', 30),
  'Martin' => array('Marie', 'Lyon', 25)
);
foreach ($personnes as $nom => $infos)
  echo "$nom : prénom={$infos[0]}, ville={$infos[1]}, âge={$infos[2]}<br>";

// 2. Clés=noms, valeurs=tableaux associatifs
$personnes2 = array(
  'Dupont' => array('prenom'=>'Paul', 'ville'=>'Paris', 'age'=>30),
  'Martin' => array('prenom'=>'Marie','ville'=>'Lyon',  'age'=>25)
);
foreach ($personnes2 as $nom => $infos)
  echo "$nom : {$infos['prenom']}, {$infos['ville']}, {$infos['age']} ans<br>";
?>
```

## ■ Exercice 3 — Date en français avec tableau associatif

```php
<?php
$date = getdate();
$jours  = array('Dimanche','Lundi','Mardi','Mercredi','Jeudi','Vendredi','Samedi');
$mois   = array(1=>'Janvier','Février','Mars','Avril','Mai','Juin',
                  'Juillet','Août','Septembre','Octobre','Novembre','Décembre');
$nomJour = $jours[$date['wday']];
$nomMois = $mois[$date['mon']];
echo "Nous sommes le $nomJour {$date['mday']} $nomMois {$date['year']}";
?>
```

## ■ Exercice 4 — Formulaire — Salutation personnalisée

```php
<!-- form.html -->
<form action='traitement.php' method='post'>
  Votre nom : <input type='text' name='nom'><br>
  Votre age : <input type='text' name='age'><br>
  <input type='submit' value='Envoyer'>
</form>

<!-- traitement.php -->
<?php
$nom = $_POST['nom'];
$age = $_POST['age'];
echo "Bonjour $nom, vous avez $age ans...";
?>
```

## ■ Exercice 5 — Calculatrice

```php
<!-- calculatrice.html -->
<form action='calcul.php' method='post'>
  <input type='text' name='a'>
  <select name='op'>
    <option value='+'>+</option>
    <option value='-'>-</option>
    <option value='*'>*</option>
    <option value='/'>/</option>
  </select>
  <input type='text' name='b'>
  <input type='submit' value='Envoyer'>
```

```
</form>

<!-- calcul.php -->
<?php
$a  = $_POST['a'];
$b  = $_POST['b'];
$op = $_POST['op'];
$res = match($op) {
  '+' => $a + $b,
  '-' => $a - $b,
  '*' => $a * $b,
  '/' => ($b != 0) ? $a / $b : 'Division par zéro',
};
echo "$a $op $b = $res";
?>
```

## ■ Exercice 6-1 — notes.php — Formulaire dynamique par nombre d'étudiants

```
<?php
if (isset($_GET['nbetu'])) {
  $nbetu = $_GET['nbetu'];
  if (is_numeric($nbetu)) {
    echo "<form action='afficheNotes_a.php' method='get'>";
    for ($i = 0; $i < $nbetu; $i++) {
      echo "<p>Etudiant ".($i+1).": nom <input type='text' name='etu$i'> ";
      echo "note <input type='text' name='note$i'></p>";
    }
    echo "<input type='hidden' name='nbetu' value='$nbetu'>";
    echo "<input type='submit' value='OK'></form>";
  } else echo 'Erreur: entrez un chiffre';
} else echo 'Erreur: indiquer le nombre d\'étudiants';
?>
```

## ■ Exercice 6-2 — afficheNotes_a.php — Tableau indicé de notes

```
<?php
$nbetu = $_GET['nbetu'];
$notes = array();
for ($i = 0; $i < $nbetu; $i++) {
  if (!empty($_GET["note$i"])) {
    $note = $_GET["note$i"];
    if (is_numeric($note) && $note >= 0 && $note <= 20)
      $notes[$i] = $note;
    else echo 'Note '.($i+1).' incorrecte.<br>';
  } else echo 'Note '.($i+1).' non renseignée.<br>';
}
echo '<table border=1><tr><th>Notes</th></tr>';
foreach ($notes as $n) echo "<tr><td>$n</td></tr>";
echo '</table>';
?>
```

## ■ Exercice 6-3 — afficheNotes_b.php — Tableau associatif nom => note

```
<?php
$nbetu = $_GET['nbetu'];
$notes = array();
for ($i = 0; $i < $nbetu; $i++) {
```

```
  if (!empty($_GET["etu$i"]) && !empty($_GET["note$i"])) {
    $nom  = $_GET["etu$i"];
    $note = $_GET["note$i"];
    if (is_numeric($note) && $note >= 0 && $note <= 20)
      $notes[$nom] = $note;
    else echo 'Note '.($i+1).' incorrecte.<br>';
  } else echo 'Données étudiant '.($i+1).' manquantes.<br>';
}
echo '<table border=1><tr><th>NOM</th><th>NOTE</th></tr>';
foreach ($notes as $nom => $note)
  echo "<tr><td>$nom</td><td>$note</td></tr>";
echo '</table>';
?>
```

## Atelier N°3 — POO

### ■ Exercice 1 — Classes Compte, CompteCourant, CompteEpargne

```
<?php
class Compte {
  protected $numero;
  protected $solde;

  public function __construct($num, $solde) {
    $this->numero = $num;
    $this->solde  = $solde;
  }
  public function crediter($montant) {
    $this->solde += $montant;
  }
  public function debiter($montant) {
    $this->solde -= $montant;
  }
  public function getSolde()  { return $this->solde; }
  public function getNumero() { return $this->numero; }
}

class CompteCourant extends Compte {
  private $decouvertAutorise;
  public function __construct($num, $solde, $dec) {
    parent::__construct($num, $solde);
    $this->decouvertAutorise = $dec;
  }
  public function getDecouvert() { return $this->decouvertAutorise; }
}

class CompteEpargne extends Compte {
  private $taux;
  public function __construct($num, $solde, $taux) {
    parent::__construct($num, $solde);
    $this->taux = $taux;
  }
  public function calculerInteret() { // signature seulement
    // return $this->solde * $this->taux;
  }
}
```

```
?>
```

### ■ Exercice 2 — Classe Etudiant — etudiant.php + test.php

**etudiant.php**

```
<?php
class Etudiant {
  private $nom;
  private $moyenne;

  public function __construct($nom, $moyenne) {
    $this->nom     = $nom;
    $this->moyenne = $moyenne;
  }
  public function getNom()    { return $this->nom; }
  public function getMoyenne(){ return $this->moyenne; }
  public function setNom($n)  { $this->nom = $n; }
  public function affiche() {
    echo $this->nom . ' — Moyenne : ' . $this->moyenne . '<br>';
  }
}
?>
```

**test.php**

```
<?php
require 'etudiant.php';

// 2 étudiants
$e1 = new Etudiant('Ali', 15.5);
$e2 = new Etudiant('Sara', 12);
$e1->setNom('Ahmed');
$e1->affiche(); // Ahmed — Moyenne : 15.5
$e2->affiche(); // Sara  — Moyenne : 12

// Tableau de 3 étudiants affiché en HTML
$tab = array(
  new Etudiant('Rami',  14),
  new Etudiant('Nour',  18),
  new Etudiant('Karim', 11)
);
echo '<table border=1><tr><th>Nom</th><th>Moyenne</th></tr>';
foreach ($tab as $etu)
  echo '<tr><td>'.$etu->getNom().'</td>'
      .'<td>'.$etu->getMoyenne().'</td></tr>';
echo '</table>';
?>
```

## Atelier N°4 — Accès BD (bdnotes)

### ■ Exercice 1 — connect.inc.php — Connexion PDO

```
<?php
$servername = 'localhost';
$username   = 'root';
$password   = '';
$dbname     = 'bdnotes';
```

```
function connexionBD() {
  global $servername, $username, $password, $dbname;
  try {
    $cnx = new PDO("mysql:host=$servername;dbname=$dbname",
                   $username, $password);
    $cnx->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    return $cnx;
  } catch (PDOException $e) {
    die('Erreur connexion: '.$e->getMessage());
  }
}
$cnx = connexionBD();
?>
```

## ☐ Exercice 2 — INSERT dans Matiere

```
<?php
require_once('connect.inc.php');
$req = "INSERT INTO Matiere(codemat, libelle, coef)
         VALUES('Mat', 'Mathematique', 0.4)";
$r = $cnx->exec($req);
if ($r) echo 'Matière ajoutée'; else echo 'Erreur';
?>
```

## ☐ Exercice 3 — UPDATE libellé de la matière ECO

```
<?php
require_once('connect.inc.php');
$req = "UPDATE Matiere SET libelle='Economie' WHERE codemat='ECO'";
$r = $cnx->exec($req);
if ($r) echo 'Modifié avec succès'; else echo 'Échec';
?>
```

## ☐ Exercice 4 — Afficher la liste des matières

```
<?php
require_once('connect.inc.php');
$stmt = $cnx->query('SELECT * FROM Matiere');
echo '<table border=1><tr><th>Code</th><th>Libellé</th><th>Coef</th></tr>';
while ($row = $stmt->fetch()) {
  echo '<tr><td>'.$row['codemat'].'</td>'
      .'<td>'.$row['libelle'].'</td>'
      .'<td>'.$row['coef'].'</td></tr>';
}
echo '</table>';
?>
```

## ☐ Exercice 5-6 — Notes avec étudiant+matière / Moyenne par étudiant

```
<?php
require_once('connect.inc.php');

// Q5: notes avec nom étudiant et libellé matière
$sql = 'SELECT E.nom, E.prenom, M.libelle, N.note
        FROM NOTATION N
        JOIN ETUDIANT E ON N.numetu = E.numetu
        JOIN EPREUVE  P ON N.numepreuve = P.numepreuve
```

```
       JOIN MATIERE  M ON P.codemat = M.codemat';
$stmt = $cnx->query($sql);
echo '<table border=1><tr><th>Nom</th><th>Prénom</th><th>Matière</th><th>Note</th></tr>';
while ($row = $stmt->fetch())
  echo "<tr><td>{$row['nom']}</td><td>{$row['prenom']}</td>"
      ."<td>{$row['libelle']}</td><td>{$row['note']}</td></tr>";
echo '</table>';

// Q6: moyenne par étudiant
$sql2 = 'SELECT E.numetu, E.nom, E.prenom, AVG(N.note) as moy
         FROM NOTATION N JOIN ETUDIANT E ON N.numetu=E.numetu
         GROUP BY E.numetu, E.nom, E.prenom';
$stmt2 = $cnx->query($sql2);
echo '<table border=1><tr><th>Num</th><th>Nom</th><th>Moyenne</th></tr>';
while ($row = $stmt2->fetch())
  echo "<tr><td>{$row['numetu']}</td><td>{$row['nom']} {$row['prenom']}</td>"
      ."<td>".round($row['moy'],2)."</td></tr>";
echo '</table>';
?>
```

## Atelier N°4 — BD Voitures (corrections des extraits fournis)

### ■ Exercice connect — connect.inc.php — bdvoitures

```
<?php
$servername = 'localhost';
$username   = 'root';
$password   = '';
$dbname     = 'bdvoitures';
try {
  $cnx = new PDO("mysql:host=$servername;dbname=$dbname",
                 $username, $password);
} catch (PDOException $e) {
  die('Connection failed: '.$e->getMessage());
}
?>
```

### ■ Exercice formulaire — Inscription propriétaire (avec requête préparée)

```
<?php
if (isset($_POST['btn'])) {
  if (!empty($_POST['nom']) && !empty($_POST['adresse'])
      && !empty($_POST['ville']) && !empty($_POST['codepostal'])) {
    $nom  = $_POST['nom'];   $prenom = $_POST['prenom'];
    $adr  = $_POST['adresse']; $ville = $_POST['ville'];
    $codep = $_POST['codepostal'];
    include 'connect.inc.php';
    $req = 'INSERT INTO proprietaire
            VALUES(NULL, :nom, :prenom, :adr, :ville, :codep)';
    $reqpre = $cnx->prepare($req);
    $reqpre->bindParam(':nom',   $nom);
    $reqpre->bindParam(':prenom',$prenom);
    $reqpre->bindParam(':adr',   $adr);
    $reqpre->bindParam(':ville', $ville);
    $reqpre->bindParam(':codep', $codep);
    $res = $reqpre->execute();
```

```
    echo $res ? 'Enregistré avec succès' : 'Erreur';
  } else echo 'Veuillez remplir tous les champs!';
}
?>
```

### ■ Exercice carte grise — Enregistrement carte grise — select dynamique

```
<?php
require 'connect.inc.php';
if (isset($_POST['enreg'])) {
  $pers     = $_POST['proprietaire'];
  $immat    = $_POST['voiture'];
  $datecarte= $_POST['datecarte'];
  $req = $cnx->prepare('INSERT INTO cartegrise VALUES(:pers, :voit, :d)');
  $req->bindParam(':pers', $pers);
  $req->bindParam(':voit', $immat);
  $req->bindParam(':d',    $datecarte);
  $r = $req->execute();
  echo $r ? 'Ajouté avec succès' : 'Erreur';
}
?>
<form action='<?= $_SERVER["PHP_SELF"] ?>' method='post'>
  Propriétaire:
  <select name='proprietaire'>
  <?php
    $res=$cnx->query('SELECT * FROM proprietaire');
    while($row=$res->fetch())
      echo "<option value='{$row['id_pers']}'>{$row['nom']} {$row['prenom']}</option>";
  ?>
  </select><br>
  Voiture:
  <select name='voiture'>
  <?php
    $res=$cnx->query('SELECT * FROM voiture');
    while($row=$res->fetch())
      echo "<option value='{$row['immat']}'>{$row['immat']}</option>";
  ?>
  </select><br>
  Date AAAA-MM-JJ: <input type='text' name='datecarte'><br>
  <input type='submit' name='enreg' value='Enregistrer'>
</form>
```

## Atelier N°4 Cookies & Session

### ■ Exercice 1 — Historique de visites avec cookie

```
<?php
$now = date('d/m/Y H:i:s');
if (!isset($_COOKIE['derniere_visite'])) {
  echo "C'est votre première visite le $now";
} else {
  $nb = isset($_COOKIE['nb_visites']) ? $_COOKIE['nb_visites'] + 1 : 2;
  echo "Vous avez consulté cette page $nb fois.<br>";
  echo 'La dernière le ' . $_COOKIE['derniere_visite'];
  setcookie('nb_visites', $nb, time()+30*24*3600);
}
```

```
setcookie('derniere_visite', $now, time()+30*24*3600);
?>
```

## ■ Exercice 2 — Choix du CSS avec cookie

```
<?php
// Choix du CSS selon cookie
if (isset($_POST['css'])) {
  setcookie('css', $_POST['css'], time()+30*24*3600);
  $css = $_POST['css'];
} elseif (isset($_COOKIE['css'])) {
  $css = $_COOKIE['css'];
} else {
  $css = 'defaut.css';
  setcookie('css', 'defaut.css', time()+30*24*3600);
}
?>
<!DOCTYPE html><html>
<head><link rel='stylesheet' href='<?= $css ?>'></head>
<body>
<h1>Bienvenue</h1>
<p>Contenu de la page...</p>
<form method='post'>
  <select name='css'>
    <option value='defaut.css'>Défaut</option>
    <option value='special.css'>Spécial</option>
  </select>
  <input type='submit' value='Appliquer'>
</form>
</body></html>
```

## ■ Exercice 3 — Sessions — ouvrir / afficher / détruire

**ouvrir_session.php**

```
<?php
session_start();
$_SESSION['prenom'] = 'Ali';
$_SESSION['role']   = 'admin';
echo 'Session ID : '   . session_id()   . '<br>';
echo 'Session name : ' . session_name() . '<br>';
echo 'Session ouverte avec succès.';
?>
```

**afficher_session.php**

```
<?php
session_start();
if (isset($_SESSION['prenom']))
  echo 'Bonjour ' . $_SESSION['prenom'] . ' (' . $_SESSION['role'] . ')';
else
  echo 'Aucune variable de session trouvée.';
?>
```

**detruire_session.php**

```
<?php
session_start();
if (session_destroy())
  echo 'Session détruite avec succès.';
```

```
else
  echo 'Erreur : impossible de détruire la session.';
?>
```

### ■ Exercice 4 — Login + vérification BD + session

```
<!-- login.html -->
<form action='verif.php' method='post'>
  Login: <input type='text' name='login'><br>
  Mot de passe: <input type='password' name='mdp'><br>
  <input type='submit' value='Se connecter'>
</form>

<!-- verif.php -->
<?php
session_start();
require 'connexion.php';
$login = $_POST['login'];
$mdp   = $_POST['mdp'];
$req   = $cnx->prepare('SELECT * FROM users WHERE login=:l AND mdp=:m');
$req->execute(array(':l'=>$login, ':m'=>md5($mdp)));
$user = $req->fetch();
if ($user) {
  $_SESSION['id']    = $user['id'];
  $_SESSION['login'] = $user['login'];
  $_SESSION['role']  = $user['role'];
  header('Location: accueil.php');
} else {
  echo 'Login ou mot de passe incorrect.';
}
?>
```

## Corrections extraits — Questions PDO (GESTION_FILM + bdvoitures)

### ■ Exercice PDO-1 — Connexion PDO — GESTION_FILM

```
<?php
$server = '100.100.100.10';
$db     = 'GESTION_FILM';
$user   = 'HarryPotter';
$pwd    = 'Alohomora';

// Connexion
$cnx = new PDO("mysql:host=$server;dbname=$db", $user, $pwd);
// Ou directement :
$cnx = new PDO('mysql:host=100.100.100.10;dbname=GESTION_FILM',
               'HarryPotter', 'Alohomora');
?>
```

### ■ Exercice PDO-2 — INSERT Réservation depuis formulaire + session

```
<?php
// Schéma: Reservation(NumRes, Nom_Film, NumPlace, NumSalle, DateRes, NumClient, EtatRes)
$nomF  = $_POST['NomFilm'];
$numS  = $_POST['NumSalle'];
$numP  = $_POST['NumPlace'];
$numC  = $_SESSION['numClient'];
```

```php
$etatR = 'non confirmée';

// Avec requête préparée (recommandé)
$sql = 'INSERT INTO Reservation(Nom_Film, NumPlace, NumSalle, DateRes, NumClient, EtatRes)
        VALUES(:nomF, :numP, :numS, NOW(), :numC, :etat)';
$req = $cnx->prepare($sql);
$req->execute(array(
  ':nomF'=>$nomF, ':numP'=>$numP, ':numS'=>$numS,
  ':numC'=>$numC, ':etat'=>$etatR
));
// NumRes est AUTO_INCREMENT, on ne le passe pas
?>
```

## ■ Exercice PDO-3 — listeReservation() — SELECT avec paramètres

```php
<?php
function listeReservation($cnx, $numClient, $dateD, $dateF) {
  $sql = 'SELECT Nom_Film, NumSalle FROM Reservation
          WHERE NumClient = :nc
          AND EtatRes = "confirmée"
          AND DateRes > :dd AND DateRes < :df';
  $req = $cnx->prepare($sql);
  $req->execute(array(':nc'=>$numClient, ':dd'=>$dateD, ':df'=>$dateF));
  echo "Mes réservations entre $dateD et $dateF :<ol>";
  while ($res = $req->fetch())
    echo '<li>'.$res['Nom_Film'].', salle '.$res['NumSalle'].'</li>';
  echo '</ol>';
}
?>
```

## ■ Exercice PDO-4 — annulerReservation() — UPDATE

```php
<?php
function annulerReservation($cnx, $num_res) {
  $sql = "UPDATE Reservation SET EtatRes='annulée' WHERE NumRes=:nr";
  $req = $cnx->prepare($sql);
  $req->execute(array(':nr' => $num_res));
}
?>
```

## ■ Exercice PDO-5 — mostWatched() — SELECT avec GROUP BY + HAVING

```php
<?php
function mostWatched($cnx, $dateD, $dateF) {
  $sql = 'SELECT Nom_Film, COUNT(*) as nb
          FROM Reservation
          WHERE DateRes > :dd AND DateRes < :df
          AND EtatRes = "confirmée"
          GROUP BY Nom_Film
          HAVING COUNT(*) > 100';
  $req = $cnx->prepare($sql);
  $req->execute(array(':dd'=>$dateD, ':df'=>$dateF));
  return $req; // retourne le PDOStatement
}
?>
```